THE ENGINEERING PROMPTING HANDBOOK

Modern Strategies for Technical Accuracy and Efficiency

1. Structural Grounding: XML Tagging

Forcing Claude to treat data and instructions separately is the #1 way to eliminate "Instruction Drift." XML tags create a logical boundary that the model respects as part of its architectural training.

> **The Rule:** Wrap raw inputs (logs, code, data) in tags to ensure the model doesn't mistake your data for a command.

Example: JSON to CSV for Excel

```
<instructions>
Convert the users inside <raw_data> into a CSV table.
- Include columns: ID, Name, Email, Status.
- If status is missing, mark as "PENDING".
</instructions>

<raw_data>
[{"id": 1, "name": "Aastha", "email": "a@example.com", "status": "ACTIVE"},
 {"id": 2, "name": "Karan", "email": "k@example.com"}]
</raw_data>
```

2. Chain-of-Thought (CoT): The "Thinking" Buffer

By default, LLMs are token-predicting machines. If you ask for a final result immediately, they "guess" the answer. Forcing a **thinking step** lets the model map the logic before committing to the output.

```
"Think step-by-step inside <thinking> tags before providing the final implementation/result."
```

Scenario: Debugging Complex API Responses

```
<task>
Analyze the following microservice trace.
```

```
1. Identify the bottleneck service.
2. Calculate the latency overhead.
3. Show your reasoning inside <thinking> tags first.
</task>
```

Benefit: Reduces calculation errors in performance metrics by up to 40%.

3. Few-Shot Prompting: Providing "Gold Standard" Examples

Don't just describe a pattern; show it. Providing 1-2 examples of the desired output style ensures the model matches your specific corporate coding or documentation standards.

Pattern	How to Prompt
Style Mirroring	"Here is an existing README. Generate a new one for Service B in the EXACT same tone and layout: [Paste Example]"
Code Conversion	"Here is how I refactor Firebird to PostgreSQL for table A. Now do the same for table B: [Paste Example]"

4. Role-Based Prompting

Assigning a persona narrows the model's probability space to a specific domain, improving the technical depth of the response.

- **Role:** "You are a Principal Software Architect specializing in Cloud Migration."
- **Role:** "You are a Technical Writer focused on high-density API documentation."
- **Role:** "You are a QA Automation Lead specializing in Playwright and Java."

5. Practical Daily Use Cases

A. Automating Technical Documentation

Turn quick notes into a professional Markdown README.

```
<system_role>Technical Writer</system_role>
<instructions>Expand these bullets into a Markdown README with headers and a
Prerequisites section.</instructions>
<notes>
- Tool: DB Sync
- Needs Java 17
- Moves data from Firebird to Postgres
```

```
- Command: java -jar sync.jar --source=... --target=...
</notes>
```

B. Data Extraction from Logs

Extract specific values from messy system logs into a structured list.

```
<instructions>
Extract all Trace_IDs and Error_Codes from the logs below.
Output as a clean bulleted list. Skip all INFO level logs.
</instructions>
<logs>
[10:00] INFO: Auth success
[10:01] ERROR: DB_TIMEOUT (ID: 99x-12)
</logs>
```

6. Accuracy Checklist for the Team

1. **Is it isolated?** (Are XML tags used for input?)
2. **Is it logical?** (Did I ask for step-by-step thinking?)
3. **Is it constrained?** (Did I say "Do not add conversational text"?)
4. **Is it contextual?** (Did I provide a role or a "few-shot" example?)